

Web Development for Whitman County Veteran Services

**Sponsored By:
Becky Buri, Veterans Service Officer**

**Faculty Mentor:
Dr. Parteek Kumar, PhD**

**By:
Logan Cribbs, Chhi Ly**



School of Electrical Engineering & Computer Science

Voiland College of Engineering and Architecture

Washington State University

Pullman, Washington

April 2026

TABLE OF CONTENTS

I. INTRODUCTION	4
II. TEAM MEMBERS & BIOS	6
III. PROJECT REQUIREMENTS SPECIFICATION	7
III.1. PROJECT STAKEHOLDERS	7
III.2. FUNCTIONAL REQUIREMENTS	7
III.2.1. EVENT MANAGEMENT	8
III.2.2. RESOURCE MANAGEMENT	8
III.2.3. USER INTERFACE	9
III.2.4. SYSTEM NAVIGATION & SEARCH	9
III.3. NON-FUNCTIONAL REQUIREMENTS	10
III.4. USE CASE (DIAGRAMS & STORIES)	11
III.5. STANDARDS	14
IV. SOFTWARE DESIGN - FROM SOLUTION APPROACH	17
IV.1. ARCHITECTURE DESIGN	17
IV.1.1. OVERVIEW	17
IV.1.2. SUBSYSTEM DECOMPOSITION	17
IV.2. DATA DESIGN	23
IV.3. USER INTERFACE DESIGN	25
IV.4. CONSTRAINTS	26
V. TEST CASE SPECIFICATIONS AND RESULTS	28
V.1. TESTING OVERVIEW	28
V.2. ENVIRONMENT REQUIREMENTS	33
V.3. TEST RESULTS	35
VI. PROJECTS AND TOOLS USED	38
VII. DESCRIPTION OF FINAL PROTOTYPE	39
VIII. SOCIAL RESPONSIBILITY AND BROADER IMPACTS	41
IX. PRODUCT DELIVERY STATUS	42
X. CONCLUSION AND FUTURE WORK	43
X.1. LIMITATIONS AND RECOMMENDATIONS	44
X.2. FUTURE WORK	45
XI. ACKNOWLEDGEMENTS	46
XII. GLOSSARY	47

XIII. REFERENCES	50
APPENDIX A - MOCKUPS AND REFINED UI EXAMPLES	52
APPENDIX B - TEAM INFORMATION	55
APPENDIX C - EXAMPLE TESTING STRATEGY REPORTING	55
APPENDIX D - PROJECT MANAGEMENT	56

I. Introduction

Supporting those who have served our country requires more than appreciation; it requires providing accessible, reliable resources that help veterans successfully transition into civilian life. Many veterans face ongoing challenges, including finding community support, navigating available services, and identifying meaningful career opportunities. This project builds upon prior development efforts aimed at addressing these needs through technology by enhancing the Whitman County Veterans Services (WCVS) e-Post website. The goal is to refine the site's structure, improve usability, and introduce features that create a more intuitive and accessible platform for veterans and their families. Central to this effort is Veteran Service Officer Becky Buri, whose work supporting the local veteran community continues to guide the direction of the project. While the current site fulfills its basic functional requirements, it presents usability challenges such as cluttered layouts, inconsistent formatting, and confusing navigation. By addressing these issues, this project seeks to transform the platform into a streamlined and user-friendly resource.

The WCVS e-Post platform operates at the intersection of community outreach and web-based resource management, serving as a centralized hub for information about events, programs, and local opportunities. Although the site contains valuable content, its presentation can hinder effective use. Problems such as uneven event tiles, inconsistent text styles, and a lack of clear visual hierarchy make it difficult for users to navigate efficiently. To improve the design, best practices from established platforms like the U.S. Department of Veterans Affairs website were considered, particularly its use of consistent navigation structures, organized layouts, and clear visual hierarchy. These principles help reduce cognitive load and improve readability, which are especially important for users with varying levels of technical familiarity. On the technical side, the platform is built using Next.js, Prisma, and SQLite, offering a lightweight and maintainable architecture compared to traditional content management systems. This stack provides developers with greater control, reduces performance and security risks associated with plugin-heavy systems, and supports future scalability through structured schema management. Deployment through Vercel further enhances the development workflow by enabling seamless integration with GitHub and simplifying continuous deployment processes.

The primary objective of this project is to continue developing a centralized and accessible platform that better serves veterans in Whitman County. Many veterans struggle to locate accurate information or remain connected to relevant community resources, making it essential that the platform prioritizes clarity and accessibility. Planned improvements focus on simplifying the layout, reducing redundant or outdated content, standardizing event listings, and improving navigation through better categorization of resources. Additionally, administrative tools will be enhanced to ensure that content remains up to date and easy to manage. These changes aim to improve both the user experience and the long-term sustainability of the platform. A more intuitive interface will allow users to quickly find critical information, while improved backend workflows will help maintain the accuracy and reliability of content over time. The success of the project will be measured by factors such as ease of navigation, user engagement, and the platform's ability to remain maintainable for future developers.

The primary client for this project is Becky Buri, who represents the needs of the Whitman County veteran community and ensures that the platform aligns with its mission of service. Additional stakeholders include veterans and their families, who rely on the site for trusted information and support; Dr. Parteek Kumar, who provides technical mentorship and guidance; the development team responsible for implementation and maintenance; and community partners such as local employers and organizations that contribute resources and opportunities. Stakeholder feedback has emphasized several key priorities, including ease of use, efficiency, professional presentation, and reliable functionality across devices. By incorporating these expectations into the design and development process, this project aims to deliver a platform that is accessible, effective, and sustainable, ultimately strengthening the support network available to veterans in the community.

II. Team Member Introduction - Bios and Project Roles

Logan Cribbs is a Computer Science student with a strong focus on full-stack development and software design. For the Whitman County Veteran e-Post Hub project, he contributed to much of the frontend development, such as the main landing page and event tile formatting/actions. Ensuring the site was responsive, accessible, and visually consistent. He also tested backend functionality and maintained database security and integrity through Prisma and SQLite. For overall project support, Logan focused on maintaining scheduled meetings with the client, tracking issues using the Kanban board, and ensuring detailed documentation. His interests include web development, UI/UX design, and building secure, user-focused applications.

Chhi Ly is a Computer Science student focused on frontend development and user interface design. He is very passionate about good software engineering practices and design patterns, and is excited to apply for this project. For this project, he helped build much of the visual layout—including the landing page, admin pages — and assisted in integrating these components with backend logic. He also worked on improving search navigation, responsiveness, and accessibility so the site remains easy to use for both veterans and administrators. Chhi contributes across the stack when needed, supporting testing, Kanban issues, and design decisions, as well as comprehensive report documentation.

III. Project Requirements Specification

This section describes the functional and non-functional requirements of the system. Each requirement is motivated by stakeholder needs and directly supports the overall mission of providing an accessible and reliable digital platform for veterans.

III.1. Project Stakeholders

- **Veterans (Primary Users):** Require an intuitive, accessible interface to find events, resources, and announcements relevant to their transition into civilian life.
- **Administrators (Content Managers):** Require secure, reliable tools to publish and update information (e.g., events, resources) quickly and accurately.
- **Veteran Service Officer (Client):** Requires the platform to reflect organizational values of accessibility, accountability, and reliability in serving the veteran community.
- **Developers/Maintainers (Secondary Stakeholders):** Require clear system requirements to ensure maintainability, scalability, and adherence to quality standards.

III.2. Functional Requirements

Below are the functional requirements listed by name with a description, source, and priority level.

III.2.1 Event Management

Name	FR-1: Create Events
Description	The system shall limit access to the create event tab to Becky, and as the admin, she should be the only person who can create an event.
Source	Specified by the client.
Priority	<u>Priority Level 0</u> : Essential and Required functionality

Name	FR-2: Set Date Ranges
Description	The system shall allow for date ranges to be set for created events.
Source	Specified by the client.

Priority	<u>Priority Level 0</u> : Essential and Required functionality
-----------------	--

Name	FR-3: Set Priority Level
Description	The system shall assign priority levels to created events based on their upcoming dates.
Source	Specified by the client.
Priority	<u>Priority Level 1</u> : Desirable Functionality

Name	FR-4: Automatic Archiving
Description	The system shall automatically archive past events, keeping them accessible in a “Past Events” section.
Source	Team recommendation after looking over the webpage.
Priority	<u>Priority Level 1</u> : Desirable Functionality

Name	FR-5: Event Flyer Downloads
Description	The system shall allow event flyers to be downloaded as PDFs for offline use.
Source	Team recommendation after looking over the webpage.
Priority	<u>Priority Level 2</u> : Stretch Feature

III.2.2 Resource Management

Name	FR-6: Resource Tab - Excel Sheet
Description	The system shall have a tab that leads to the Excel sheet Becky has created, containing all the updated contact information and correct offices.
Source	Specified by the client.

Priority	<u>Priority Level 0</u> : Essential and Required Functionality
-----------------	--

Name	FR-7: Mann-Grandstaff Link
Description	The system shall open up the Mann-Grandstaff page when the corresponding link is clicked.
Source	Specified by the client.
Priority	<u>Priority Level 0</u> : Essential and Required Functionality

III.2.3 User Interface

Name	FR-8: Uniform Event Tiles
Description	The system shall display tiles with uniform events on the main page.
Source	Specified by the client.
Priority	<u>Priority Level 0</u> : Essential and Required Functionality

Name	FR-9: Event Image Display
Description	The system shall display the image of a clicked-on event tile on the same webpage, without opening a new tab.
Source	Specified by the client.
Priority	<u>Priority Level 1</u> : Desirable Functionality

III.2.4 System Navigation & Search

Name	FR-10: Email Link
Description	The system shall display a new email draft to Becky when the email link is clicked, rather than redirecting to the 365 sign-in page.
Source	Team recommendation after looking over the webpage.

Priority	<u>Priority Level 0</u> : Essential and Required Functionality
-----------------	--

Name	FR-11: Search Functionality
Description	The system shall include a search function that allows users to quickly locate resources, events, or contact information by keyword.
Source	Specified by the client.
Priority	<u>Priority Level 1</u> : Desirable Functionality

III.3. Non-Functional Requirements

The following are non-functional requirements outlining the qualities the website and backend must uphold. These objectives ensure that the system remains reliable, accessible, and user-friendly, even though they do not directly define its functional behavior.

- **Usability and Design:** The system shall present a clean and simple layout with properly categorized information, responsive headers and footers, consistent formatting, and readable font sizes (14–16 px for body text, 20–24 px for headings). The design shall avoid horizontal scrolling at 100 percent zoom and maintain sufficient color contrast in compliance with WCAG 2.1 AA standards.
- **Performance:** The system shall achieve page load times of less than or equal to three seconds on standard broadband connections and update newly published events, resources, and announcements within five minutes.
- **Reliability:** The system shall maintain 99% uptime on a monthly basis to ensure veterans have consistent access to time-sensitive resources.
- **Accessibility:** The system shall ensure that users, including those with disabilities, can navigate and interact with the website by adhering to accessibility standards and guidelines.
- **Accountability:** The system shall log all administrative actions (e.g., event creation) to support transparency, auditing, and oversight.

III.4. Use Cases (Diagrams & Stories)

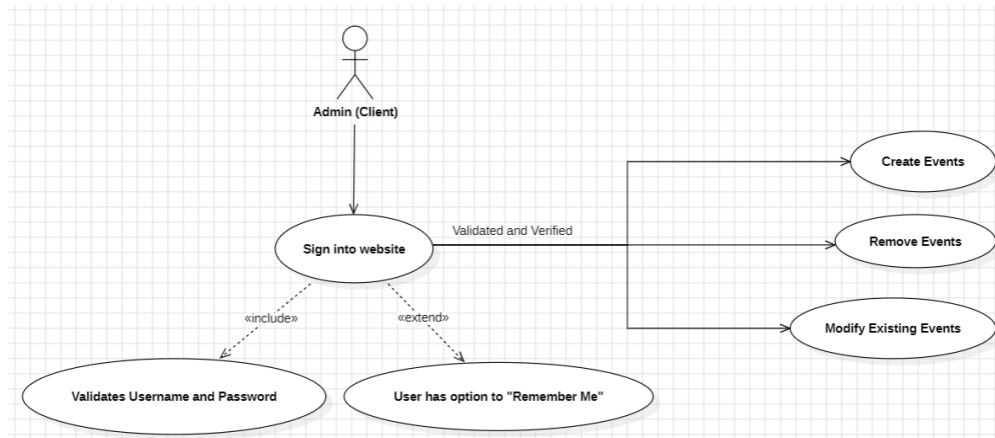


Figure 1. Use Case Diagram illustrating Admin sign-in and the Actions that the user is authorized to take.

UC-1: Admin Sign-in to Edit Events

Actors	Admin/Client
Pre-condition	The user is on the Admin sign-in page.
Post-condition	The user is successfully authorized and granted access to event management features after credential verification.
Basic Path	1. The user navigates to the Admin sign-in Page. 2. The system displays the login form with username/email and password fields. 3. The user enters valid credentials. 4. Click the "Sign In" button. 5. The system verifies the credentials against stored records. 6. Upon successful verification, the system establishes an authenticated session. 7. The system redirects the user to the Admin dashboard where event editing options are available.
Alternate Paths	3a. The user enters incorrect credentials. 4a. The user selects "Forget Password". 5a. The credentials are invalid and return to step 1.

US-01:

"As an admin, I want a streamlined event creation option so that I am not filling out unneeded information."

Feature: Admin Event Management.

Scenario: User is signed in and creates an event.

Given the user is authenticated as an admin.

And the user is on the “Create Event” form.

Create an event by filling only the required fields and relevant details.

And the user submits the form.

Optional fields not shown are saved as null.

And the user sees a confirmation with the event tile.

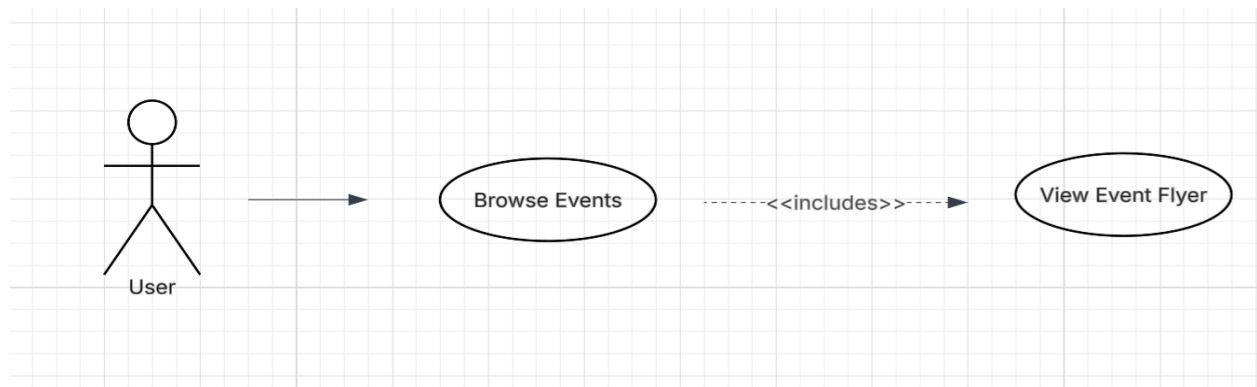


Figure 2. Use Case Diagram illustrating User interaction to view an event flyer from browsing the event tiles grid.

UC-2: View Uniform Event Flyers

Actors	General User
Pre-condition	The user is on the landing page, and the event tiles have loaded.
Post-condition	The selected event flyer is displayed on the same page, in a uniform and consistent size.
Basic Path	1. The user scrolls to the event tiles section on the main page. 2. The user clicks an event tile. 3. The system retrieves the flyer associated with the selected event. 4. The system displays the flyer on the same webpage using a fixed bounding frame to enforce uniform size. 5. The system preserves aspect ratio (fit/contain) to avoid distortion. 6. The tiles/flyer area remains uniform and readable.
Alternate Paths	2a. The system shows a placeholder (“Flyer unavailable”) within the same fixed frame to preserve uniformity. 4b. The system shows a loading skeleton within the fixed dimensions until the flyer renders.

US-02:

“As a community veteran, I want the event flyers to be uniform so that they are readable.”

Feature: Event flyers display uniformly.

Scenario: Event tiles load flyers in a uniform format.

Given that I have scrolled down to the event tiles on the webpage.

When I click on different events, the flyers display at the same image size.

Then all the event tiles appear uniform on the webpage.

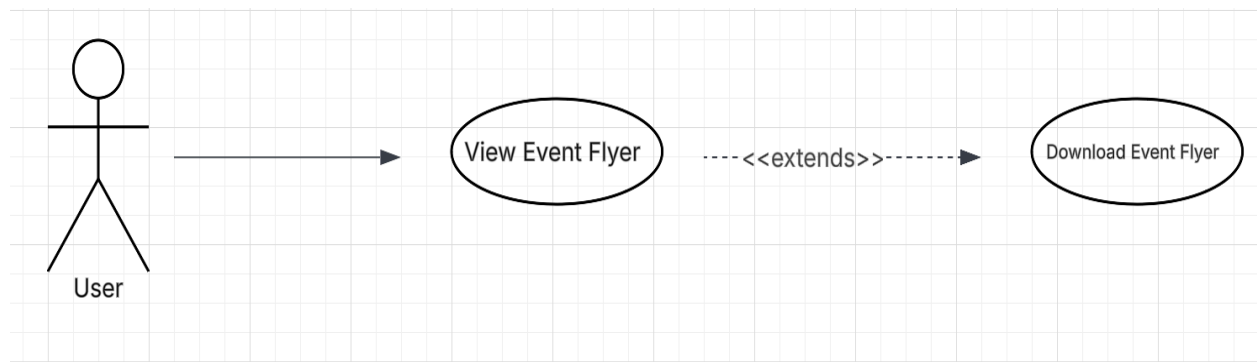


Figure 3. Use case diagram illustrating the User being able to download a PDF of the event flyer.

UC-3:Download Event Flyers

Actors	General User
Pre-condition	The user is viewing the event details, and the flyer is visible.
Post-condition	The flyer is successfully downloaded and can be opened or shared offline.
Basic Path	1. The user navigates to the event details page. 2. The system displays the event flyer on the page. 3. The user clicks the “Download Flyer (PDF)” button. 4. The system generates a PDF version of the flyer. 5. The system initiates the download of the flyer to the user’s device. 6. The user opens the file, and the PDF is readable.
Alternate Paths	2a. If the flyer is not available, the system shows a “No flyer available” message and disables the download option. 4a. The system displays an error message and prompts the user to retry. 5a. The system fails to complete the download and notifies the user to check their connection.

--	--

US-03:

“As a veteran, I want event flyers to be downloadable so that I can print or share them offline.”

Feature: Event Flyer Downloads.

Scenario: Download event flyer as PDF.

Given that I am viewing an event details page.

And the event flyer is visible on the page.

When I click the “Download Flyer (PDF)” button.

Then the system generates a PDF of the flyer.

And the file begins downloading to my device.

And the downloaded file opens successfully and is readable.

III.5. Standards

The Whitman County Veterans Services (WCVS) e-Post Hub project adheres to established industry, accessibility, documentation, and ethical standards to ensure the system is reliable, secure, usable, and professionally maintainable. These standards guided the formulation of both functional and non-functional requirements and serve as benchmarks for verification throughout development and testing.

Accessibility Standards

The system is designed in compliance with the Web Content Accessibility Guidelines (WCAG) 2.1 AA, ensuring equitable access for users with disabilities. This includes:

- Proper color contrast ratios (minimum 4.5:1)
- Scalable typography and readable font sizes
- Keyboard navigation support
- Semantic HTML and ARIA labeling for screen readers
- Descriptive alt text for images and event flyers

Accessibility compliance directly supports the project’s mission of serving veterans of varied ages and physical ability levels.

Software Requirements & Documentation Standards

The structure and traceability of the system's requirements follow the **IEEE 830 Software Requirements Specification (SRS)** standard. This ensures that:

- All functional (FR) and non-functional requirements (NFR) are clearly defined
- Each requirement maps directly to user stories, use cases, and verification methods
- The Requirements Traceability Matrix maintains end-to-end accountability from specification to testing

Additionally, testing documentation aligns with the **IEEE 829 Test Documentation Standard**, providing consistent formatting for test plans, test cases, and defect reports.

Data Privacy & Security Standards

Although the system does not collect personal data from general users, it follows core principles from the **General Data Protection Regulation (GDPR)** for ethical data handling. Security measures include:

- Encrypted password storage using *bcrypt* hashing
- HTTPS-only secure data transmission
- Restricted role-based access for administrative users
- Protection against unauthorized access to event management features

These practices ensure responsible handling of administrative credentials and system integrity.

Professional & Ethical Standards

The development team follows the ACM Code of Ethics and Professional Conduct, which promotes:

- Respect for user privacy
- Inclusive and accessible system design
- Transparent software behavior
- Responsible decision-making in public service software development

This standard reinforces the ethical responsibility associated with building software for a community-focused government organization.

Frontend & Web Standards

The system follows established W3C HTML5, CSS3, and modern JavaScript (ES6+) standards, ensuring:

- Cross-browser compatibility (Chrome, Edge, Firefox, Safari)
- Responsive design behavior across mobile, tablet, and desktop
- Clean semantic structure for long-term maintainability

IV. Software Design - From Solution Approach

IV.1. Architecture Design

IV.1.1. Overview

The WCVS e-Post Hub uses a modular and layered architecture based on the client-server model, which divides the system into independent layers that communicate through clearly defined interfaces. This structure improves maintainability and testing efficiency, allowing frontend and backend components to evolve independently.

The architectural pattern adopted is a Model-View-Controller (MVC) variation implemented through Next.js, where:

- The view is handled by React components and Tailwind CSS[8] layouts handle the View.
- The controller logic is implemented in Next.js API routes that process user actions and inputs.
- The model is defined through Prisma's schema, managing data persistence and integrity.

This pattern fits the project well because it supports high cohesion within each layer and low coupling between the frontend and backend. It allows developers to work on UI improvements, event management features, or database enhancements independently, minimizing merge conflicts and simplifying debugging.

IV.1.2. System Decomposition

The system is decomposed into five major subsystems:

1. **Next.js Frontend**
2. **API Routes / Server Logic**
3. **Authentication Subsystem**
4. **Prisma Database Layer**
5. **File Management and Flyer Handler**

Below is a UML component diagram that illustrates the system's subsystem decomposition for the WCVS Veteran e-Post Hub webpage. The diagram shows how the major components interact within the system architecture. Each subsystem is described in detail below, including its responsibilities, concepts, interfaces, and role in the overall architecture.

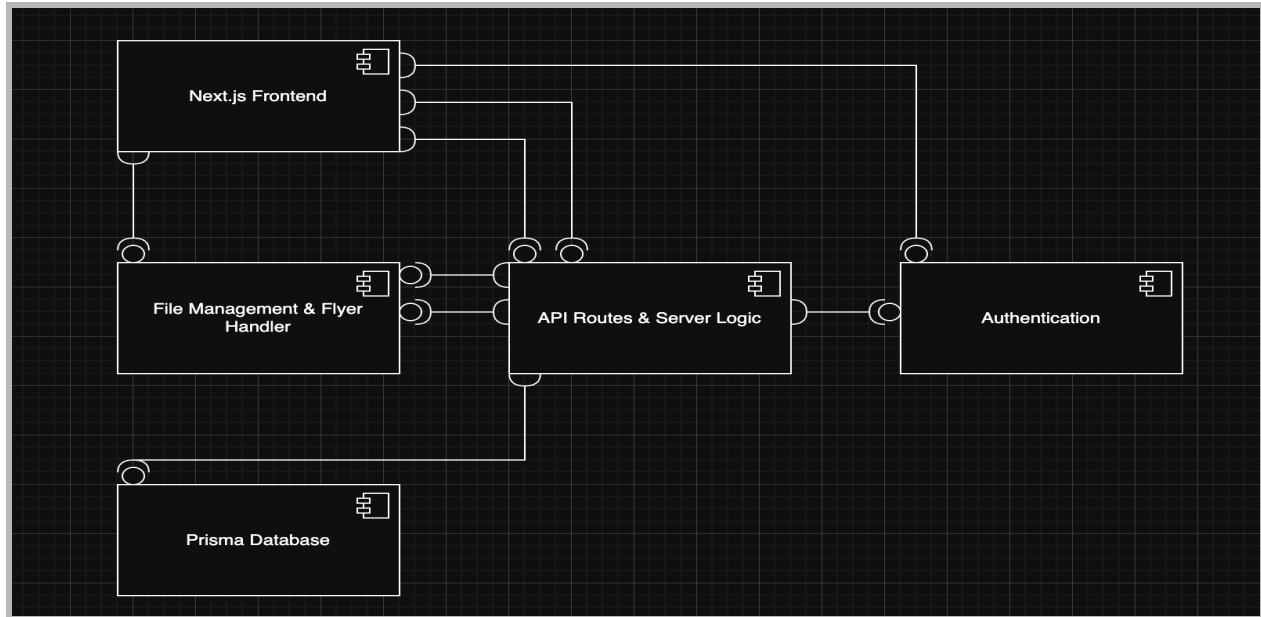


Figure 4. Subsystem Decomposition

IV.1.2.a. Next.js Frontend

Description

The Next.js Frontend represents the user-facing interface of the e-Post Hub. It is built using React components, Tailwind CSS, and HeroUI. The frontend is responsible for rendering dynamic pages, handling user interactions, and providing a consistent design across all screen sizes[8].

It includes the following key pages:

- **Home Page:** General introduction and quick access to current events.
- **Events Page:** Displays event listings with thumbnails, descriptions, and flyer download buttons.
- **Admin Login & Dashboard:** Allows authorized users to log in and manage events.

Concepts and Algorithms Generated

The frontend uses a component-based architecture for modularity. Components such as EventCard, Navbar, and EventForm are reusable across multiple pages. The data flow follows React's state management principles, using local state for form handling and the SWR (stale-while-revalidate) library for real-time event data updates.

Dynamic routing is implemented through Next.js's file-based routing, which automatically maps URLs to page components. The decision to use Next.js instead of plain React was made because it supports server-side rendering (SSR) for improved SEO and faster load times,

particularly for the public-facing event pages.

Interface Description

Services Provided:

Service Name	Provided To	Description
Render Pages	Users / Admins	Displays interactive pages (Home, Events, Admin) using responsive layouts.
Send Form Data	API Routes	Sends user input (login credentials, event forms) to backend endpoints.
Display Event Data	Users	Fetches event data and renders it as formatted cards or tables.

Services Required:

- Event data from API Routes
- Authentication tokens from Auth Subsystem
- Flyer URLs from File Handler

IV.1.2.b. API Routes / Server Logic

Description

The API Routes act as the bridge between the frontend and the backend. Implemented through Next.js's built-in /api directory, these routes handle all communication, data validation, and logic for *CRUD* operations.

Key endpoints include:

- /api/events → Get all approved events.
- /api/event/create → Add new event (admin only).
- /api/event/edit and /api/event/delete → Update or remove existing events.
- /api/auth/login → Verify user credentials.

Concepts and Algorithms Generated

Each route follows a RESTful design pattern, maintaining consistency across the API. The routes use *Zod* for input validation to ensure proper data structure before updating the database. Error handling returns clear HTTP codes and JSON responses for frontend interpretation.

Middleware is used to validate authentication tokens and restrict admin operations. This provides a security layer without requiring an external server or framework.

Interface Description

Services Provided

Service Name	Provided To	Description
Fetch Events	Frontend	Retrieves all event data and returns JSON for rendering.
Create Event	Admin Dashboard	Adds new event details to the database after validation.
Update Event	Admin Dashboard	Edits event information and updates associated flyers.
Delete Event	Admin Dashboard	Removes events and cleans up linked flyer references.

Services Required

- **Database access** (via Prisma)
- **User verification** (from Auth Subsystem)
- **Flyer upload/delete** (from File Handler)

IV.1.2.c. Authentication Subsystem

Description

The Authentication subsystem manages secure logins and session control for administrative users. It ensures that only authorized personnel can modify event data or upload new flyers.

Concepts and Algorithms Generated

The subsystem uses *bcrypt* to hash and verify passwords. Authentication requests are processed through `/api/auth/login`, where user credentials are compared to hashed records in the database. Upon successful login, NextAuth.js (or JWT tokens) maintains the session securely using HTTP-only cookies.

This design prioritizes simplicity and portability while still adhering to secure authentication practices.

Interface Description

Services Provided

Service Name	Provided To	Description
Validate Login	API / Frontend	Authenticates user credentials and issues a session token.
Authorize Access	API Routes	Confirms token validity for admin-only operations.
Logout Session	Frontend	Ends current session and clears cookie/token.

Services Required

- **User record retrieval** (from Prisma Database Layer)

IV.1.2.d. Prisma Database Layer

Description

The Prisma Database Layer acts as the system's data management core. It provides a type-safe interface for interacting with the underlying SQLite database. Prisma automatically generates functions to perform *CRUD* operations based on the schema[10].

Concepts and Algorithms Generated

The schema defines three main models:

- **User:** Contains username, hashed password, and role (admin).
- **Event:** Stores title, description, date, and flyer URL.
- **ResourceLink:** Stores external resource references for veterans.

Migrations are handled automatically through Prisma CLI commands. During deployment, the same schema can be easily adapted for PostgreSQL on Vercel to support concurrent connections.

Interface Description

Services Provided

Service Name	Provided To	Description
Retrieve Events	API Routes	Returns all event entries for frontend display.
Insert Event	API Routes	Adds new event records to the database.
Update Event	API Routes	Modifies event entries (title, date, or flyer).
Delete Event	API Routes	Removes event data when deleted by the admin.
Authenticate User	Auth Subsystem	Retrieves user information and hashed credentials.

Services Required

- **None** (accessed internally through Prisma Client).

IV.1.2.e. File Management & File Handler

Description

This subsystem manages all flyer and document uploads. It ensures each event's flyer maintains uniform formatting and downloadable functionality. Files are temporarily processed and linked to their corresponding event database entry.

Concepts and Algorithms Generated

File uploads are validated for type (.pdf, .jpg, .png) and size before being stored. Each upload triggers a function to update the associated database record with a public URL reference. Flyer downloads use standard HTTP GET requests with built-in browser PDF handling.

For future scalability, the design can integrate cloud storage such as AWS S3 or Vercel Blob to replace local handling if needed.

Interface Description

Services Provided

Service Name	Provided To	Description
Upload Flyer	Admin Dashboard / API	Accepts uploaded event flyer and returns public link.
Retrieve Flyer	Frontend	Provides download or preview capability for users.
Delete Flyer	Admin Dashboard	Removes the flyer file and clears the record on deletion.

Services Required

- **Database Update** (via Prisma)
- **Auth Verification** (via Authentication Subsystem)

IV.2. Data Design

The data design for the Whitman County Veteran Services (WCVS) e-Post Hub focuses on creating a structured, efficient, and maintainable schema to support both administrative and user-facing functionality. The system employs Prisma ORM to define the schema and interact with a lightweight SQLite database. This design allows for clear entity relationships, type safety, and consistent data validation throughout development.

1. Database Overview

The application follows a relational data model, where each table corresponds to a logical entity within the system. The schema supports *CRUD* operations (Create, Read, Update, Delete) across event and user data. By using Prisma's schema definition language, developers can easily synchronize the database with code-level models and maintain referential integrity.

2. Entities and Relationships

The major Entities in the system are:

- **User** – Stores information about administrators, including credentials and access privileges.
- **Event** – Represents community events, including titles, descriptions, and associated flyers.

- **Flyer** – Contains metadata about uploaded PDF flyers linked to specific events.
- **Resource** – Stores external links and references to veteran service organizations.
- **SessionToken** - Tracks active login sessions to maintain secure authentication with appropriate authorized actions.

Entity Relationships:

- One **User** can create multiple **Events**.
- Each **Event** has one corresponding **Flyer** file.
- All **Resources** are independent and publicly accessible.
- One **User** can have multiple **SessionTokens**.

These relationships promote data normalization and minimize redundancy, ensuring each entity has a single, well-defined purpose within the database.

3. Data Validation and Integrity

All data undergoes validation both at the application and schema levels:

- **Zod** validation enforces correct data types and formats on API input.
- **Prisma** constraint ensures referential integrity between various entities.
- Timestamps ('createdAt', 'updatedAt') are automatically managed and maintained by **Prisma**.
- Input sanitization prevents injection or malformed requests.

4. Sample Prisma Schematic

```
model User {
  id          Int           @id @default(autoincrement())
  username    String        @unique
  passwordHash String
  role        Role          @default(ADMIN)
  events      Event[]
  sessions    SessionToken[]
  createdAt   DateTime       @default(now())
  updatedAt   DateTime       @updatedAt
}
```

Figure 5. Sample schema for User (Admin)

```
model Flyer {
  id          Int           @id @default(autoincrement())
  filename    String
  path        String
  uploadedAt  DateTime       @default(now())
  event       Event          @relation(fields: [eventId], references: [id])
  eventId     Int            @unique
}
```


Figure 6. Sample schema for Flyer

```
model Resource {  
  id      Int      @id @default(autoincrement())  
  title   String  
  link    String  
  category String? //nullable  
  createdAt DateTime @default(now())  
}
```

Figure 7. Sample schema for Resource

IV.3. User Interface Design

The user interface (UI) for the WCVS Veteran e-Post hub is designed to prioritize simplicity, accessibility, and consistency for both veterans and administrators. Built using Next.js, React components, and Tailwind CSS, the interface emphasizes a seamless and responsive experience across devices.

1. Design Principles

- **Consistency:** Uses Tailwind CSS and HeroUI for uniform design.
- **Accessibility:** Follows WCVS color palette and contrast guidelines.
- **Responsiveness:** Adapts across mobile, tablet, and desktop views.
- **Simplicity:** Reduces clutter and uses clear navigation for veterans unfamiliar with tech.

2. Key Pages and Components

- Home Page
 - Rebranding the look of the Veteran e-Post hub landing page.
 - Shows featured or upcoming events. Layout is more 'Intuitive'.
 - Contains directory/navigation links: Home, Resources, Contact, About.
- Events Page
 - Lists events as interactable cards with viewable thumbnail images.
 - Each card links to full event details that will no longer create a new tab.
- Admin Dashboard
 - Login form (username + password) strictly for Admin only.
 - After login: Contains *CRUD* panel for event management.
Features include:
 - Add new event tile with description, and flyer upload.
 - Edit or delete existing events.
 - Live preview of event list.

3. Navigation Flow

- Users navigate from the home page to view events or access resources.

- Administrators log in to the dashboard to perform *CRUD* operations on event data.
- The navigation bar and footer remain persistent throughout all pages for consistency.

4. Visual Prototypes

Figma wireframes and mockups (*see Appendix section*) illustrate the envisioned final layout, color scheme, and responsive behavior of the WCVS e-Post Hub interface.

These prototypes provide a tangible representation of the design direction and serve as a guiding reference throughout the development process.

The visual design emphasizes clarity, accessibility, and adherence to WCVS branding standards. Client feedback highlighted that the current layout provides a strong foundation for future enhancements, offering both aesthetic appeal and functional extensibility as the platform evolves.

IV.4. Constraints

1. Technical Constraints

- Framework Limitations:** The system architecture is bound to the Next.js 14 framework and React 18 runtime, limiting backend flexibility compared to a full Node.js Express server.
- Database Choice:** The initial deployment uses SQLite for simplicity and cost-efficiency; however, concurrent multi-admin editing is limited due to SQLite's file-based locking mechanism.
- Hosting Environment:** Deployment on Vercel imposes execution-time limits on serverless functions (up to 10 seconds for free tier) and storage restrictions that may affect future scalability.

2. Environmental Constraints

- Browser Compatibility:** The UI must perform consistently across modern browsers (Chrome, Edge, Firefox, Safari). Legacy browsers (IE 11 and below) are not supported.
- Responsive Design:** All pages must auto-scale and remain fully functional across screen sizes from 320 px (mobile) to 1920 px (desktop).
- Deployment Dependence:** System availability is tied to Vercel's uptime and GitHub synchronization for continuous deployment.

3. Operational Constraints

- Administrative Access:** Only Whitman County Veteran Services authorized admins can log in to create, edit, or delete events.
- Data Backup:** SQLite database backups must be scheduled manually before each major release.

4. Security and Privacy Constraints

- a. **Password Storage:** All passwords must be hashed with *bcrypt* before database insertion; plaintext passwords are strictly prohibited.
- b. **Transport Security:** All requests must be served over HTTPS via Vercel's built-in SSL encryption.
- c. **Data Retention:** Personally identifiable information is not collected from users beyond admin credentials.

5. Project Management Constraints

- a. **Timeline:** The prototype must be completed by December 5th, 2025, as a deliverable for the WSU capstone course.
- b. **Team Size:** Development is limited to two contributors, restricting parallel feature development and QA capacity.
- c. **Budget:** The project must operate within zero-cost or student discount tiers for all tools (Next.js, Vercel, GitHub, Prisma Cloud).

V. Test Case Specifications and Results

V.1. Testing Overview

Testing for the Veteran e-Post Hub follows a structured, bottom-up pipelined approach, ensuring that each layer of the system is verified independently before integration with the next.

Automated testing with Vitest establishes component-level reliability and correctness, while browser-driven Selenium scripts validate end-to-end workflows from a user's perspective.

The overall testing sequence proceeds as follows:

1. Run Vitest unit tests during local development to verify individual components and logic.
2. Perform Vitest integration tests after merging pre-built and new interface features to ensure data consistency and communication across modules.
3. Execute Selenium end-to-end tests in a controlled browser environment to simulate real user interactions and confirm system behavior.
4. Validate final behavior through User Acceptance Testing (UAT) in collaboration with Becky Buri, the Veterans Service Officer, to ensure the application meets client expectations and usability standards.

V.1.1. Testing Tools

- **Vitest** - a fast JavaScript testing framework used for unit and integration testing of the frontend codebase.
- **Selenium WebDriver** - used for system and user-acceptance testing. It automates browsers to replicate real user actions such as navigation, form submission, and event lookup, ensuring the UI behaves as expected across devices and screen sizes.
- **Manual Exploratory Testing** - complements automation by verifying accessibility, visual consistency, and responsive design on multiple screen dimensions.

V.1.2. Testing Workflow & Flow of Action

1. **Plan:** Derive test cases directly from the Software Requirements Specification (SRS) and user stories to ensure all functional and non-functional requirements are covered.
2. **Develop:** Create new test scripts using **Vitest** and **Selenium** to verify functional logic, interface behavior, and workflow accuracy. Reuse or modify existing test cases when implementing new or updated code to maintain continuity across versions.

3. **Execute:** Run automated test suites in both local development and staging environments to validate consistent behavior across builds.
4. **Record:** Capture test results, screenshots, and logs. Document any observed defects or anomalies in the shared issue-tracking system for team review.
5. **Fix and Re-test:** Address identified issues, apply corrective patches, and re-run affected tests to confirm proper resolution.
6. **Accept:** Once all critical and high-priority test cases have passed successfully, submit the build for **User Acceptance Testing (UAT)** and formal client sign-off.

V.1.3. Continuous Integration/ Continuous Delivery (CI/CD)

A lightweight, manual integration workflow is implemented in place of a fully automated CI/CD pipeline. This approach provides consistency and quality assurance while remaining manageable for the small development team.

The workflow proceeds as follows:

1. **Vitest test suites** are executed locally before every commit or pull request to ensure new code does not introduce regressions.
2. **Selenium end-to-end tests** are manually executed before deploying to the staging environment, validating complete user workflows in a real browser context.
3. **Only builds that pass both Vitest and Selenium tests** are merged into the main working branch and prepared for final deployment.

Although automation tools such as GitHub Actions could potentially be integrated in future iterations, the current manual approach effectively enforces testing discipline and quality control within the project's scope and resources.

V.1.4. Reporting and Documentation

All test artifacts and results will be archived in the project repository's /test directory, including Vitest console outputs, Selenium execution logs, and captured screenshots. These records provide traceability for all test activities and support the reproducibility of results.

Bugs and defects are tracked through issue *tickets* on the team's Kanban board. Each ticket documents the related test case, observed failure conditions, root cause (if identified), and resolution status.

Upon completion of all testing phases, a summary testing report and acceptance checklist will be compiled and submitted for final client approval[14].

V.1.5. Testing and Development Rationale

This dual-tool testing strategy provides both granular verification and end-to-end confidence while remaining lightweight and practical for a small development team.

Vitest delivers rapid feedback on new code changes, ensuring component-level accuracy and stability during development.

Selenium, in contrast, validates that existing features continue to function correctly alongside new implementations—preserving overall system integrity and ensuring the deployed application aligns with user expectations.

The following section describes the detailed plan for conducting all test activities for the Veteran e-Post Hub. These activities follow the sequential strategy outlined earlier and use test cases derived directly from the WRS, user stories, and sprint deliverables for modules such as the Events system, Seasonal UI theming, Veteran Resource tiles, and the Administrative Dashboard.

Testing will proceed in a bottom-up approach, starting with React components and utility functions, gradually incorporating API routes, and culminating in full end-to-end system validation through Selenium.

V.1.6. Unit Testing

Unit testing will be conducted using Vitest, targeting the smallest testable components within the project. In the context of the e-Post Hub, this includes:

- UI components such as the rotating top banner, resource tiles, event cards, form fields, seasonal theme toggles, and the Veteran Health Care / Resource Guide navigation links.
- Utility logic such as date formatting, seasonal theme calculation, event sorting, file-upload validation, and admin-role checks.
- Prisma-backed service functions when run in isolated mocked environments, including event creation, fetch calls, and resource retrieval.

Sequence of execution:

1. Identify target component/logic.
2. Stub external dependencies such as Prisma queries or API routes using mocks.
3. Execute Vitest tests locally after each major component update to ensure no regressions.
4. Verify rendering output, expected props behavior, and error handling.
5. Record failures, revise the test or component code, and re-run the suite.
6. Mark the unit as verified before integrating it with larger modules.

This phase ensures that the core frontend pieces behave predictably and that backend logic returning data to the UI remains stable before integration begins.

V.1.7. Integration Testing

Integration testing verifies how multiple modules interact—particularly critical in a system like the Veteran e-Post Hub, where event data, resource listings, and admin tools flow through shared API routes and the Prisma layer.

Key areas of integration include:

- **Admin Event Management → Database → User Event Display**
Ensures events created/edited by the VSO appear correctly on the public resources/events page.
- **Seasonal UI Theme System → Global Layout → Component Styling**
Confirms that seasonal color palettes propagate consistently across headers, tiles, banners, and forms.
- **Resource Guide Data → Tile Rendering → Navigation Links**
Ensures that updated resources show correctly with consistent formatting and link behavior.

Sequence:

1. Run unit tests first to ensure all underlying components are stable.
2. Combine related modules such as the Admin dashboard + Events API + Event Card display.
3. Use Vitest integration test cases with mocked database layers to simulate multi-step flows.
4. Execute tests on shared states, such as season changes, event prioritization, and role-based routing.
5. Log defects related to cross-module communication.
6. Re-test once fixes are applied.

Integration testing ensures that the system behaves cohesively, not merely that individual components work in isolation.

V.1.8. System Testing

System testing evaluates the entire Veteran e-Post Hub as one integrated application. Selenium WebDriver will be used to simulate realistic user behavior such as navigating the home page, interacting with the rotating banner, opening resource tiles, submitting forms, and performing admin actions.

V.1.9. Functional Testing

Functional system tests will verify that all features described in the WRS operate as intended. Key tests include:

- **Viewing Events:** Ensure events load dynamically, display correct formatting, and prioritize the correct banners.
- **Resource Access:** Verify that tiles (Veteran Health Care, Resource Guide, WCVS Links) open the correct destinations.
- **Form Submission:** Validate file uploads, required field checks, error messages, and success states.
- **Admin Dashboard Tools:** Confirm that the VSO can add, remove, or update events and resources.
- **Seasonal Theme Shift:** Verify that UI themes change appropriately based on date and configuration.
- **Accessibility Tests:** Ensure proper keyboard navigation, alt text, aria labels, and screen-reader friendliness.

Functional tests will be selected based on the high likelihood of uncovering failures in core workflows used daily by veterans and the VSO.

V.1.10. Performance Testing

Performance tests will evaluate responsiveness and load behavior, including:

- **Page Load Speed:** Ensuring the home page (banner + tiles + rotating events) loads within acceptable limits.
- **Database Response Times:** Measuring the performance of event queries via Prisma when the dataset grows.
- **Stress Testing:**
 - Simulating large numbers of upcoming events.
 - High-frequency admin dashboard updates.
 - Multiple concurrent resource requests.

The goal is to confirm non-functional requirements such as responsiveness, stability, and scalability under conditions beyond typical use.

V.1.11. User Acceptance Testing (UAT)

User Acceptance Testing will be performed collaboratively with Becky Buri, the project sponsor, and VSO. This phase ensures that the system aligns with the practical needs of Whitman County Veteran Services.

UAT activities include:

- Demonstrating the admin dashboard workflow for event management.
- Reviewing the seasonal theme accuracy and usability of the updated designs.
- Confirming that resource navigation matches the expected paths and naming conventions.
- Validating the rotating top banner for clarity, correctness, and usefulness.
- Reviewing accessibility, readability, and overall ease of use.

Only after the client, Becky Buri, approves each critical feature will the system be marked ready for deployment.

V.2. Environment Requirements

The testing environment for the Veteran e-Post Hub must support browser-based testing, database interactions through Prisma, and execution of automated test tools such as Vitest and Selenium. The environment will replicate the production conditions as closely as possible to ensure reliable and consistent results.

V.2.1. Hardware Requirements

Minimum Hardware (Developer & Test Machines)

- **CPU:** Dual-core processor or higher.
- **RAM:** 8 GB minimum (16 GB recommended for concurrent Selenium runs).
- **Storage:** 10 GB free for local node modules, logs, build files, and database snapshots.
- **Display:** 1080p resolution for UI validation.
- **Network:** Stable broadband connection for Vercel preview deployments and repository access.

Selenium Test Machines

- Should support modern browsers with WebDriver support.
- May run tests locally or inside lightweight virtual environments.

V.2.2. System & Software Requirements

Operating Systems

- macOS 13+
- Windows 10/11
- Ubuntu 20.04+

Required Software

- **Node.js v18+** (for Next.js and Vitest).
- **npm or yarn** for dependency management.
- **Next.js 14+** (core framework).
- **Prisma ORM** with SQLite/PostgreSQL drivers.
- **Selenium WebDriver**.
- **Chrome or Firefox Browsers**.
- **VS Code or JetBrains editors** for development.

Runtime Dependencies

- **TailwindCSS** for UI testing consistency.
- **Prisma Client** for database interactions.
- **React Testing Library** (used alongside Vitest).
- **Vercel Deployment Preview** for cross-browser user acceptance demonstrations.

V.2.3. Database Requirements

During testing, the Prisma ORM will use:

- **SQLite (local)** for fast unit and integration testing
- **PostgreSQL (staging)** for system-level testing, reflecting production-like conditions

Database states will be reset before each test cycle using (prisma migrate reset) to ensure deterministic test outcomes.

V.2.4. Network & Communication Requirements

- Access to the GitHub repository for test scripts and logs.
- Access to Vercel preview URLs for UAT.
- Stable connection for Selenium remote driver execution (if used).
- Standard HTTP/HTTPS communication for API calls.

V.2.5. Test Tools

- **Vitest** – unit & integration testing.

- **Selenium WebDriver** – system & UAT automation.
- **React Testing Library** – DOM interaction testing.
- **Browser DevTools** – performance profiling.
- **GitHub Kanban Board** – defect tracking, product and sprint backlogs.

V.2.6. Mode of Usage

Testing will be conducted in a stand-alone developer environment for unit and integration testing, and in a staging server environment for complete system testing and UAT. Browser tests will be run using both desktop and mobile resolutions to reflect the application's responsive design requirements.

V.3. Test Results

This section presents the outcomes of the testing process conducted on the Veteran e-Post Hub system. Test cases were derived directly from the functional and non-functional requirements outlined in the Software Requirements Specification (SRS) and executed using a combination of Vitest for unit and integration testing, Selenium for system-level testing, and manual validation for usability and accessibility checks.

The objective of this testing phase was to verify that all core functionalities operate as intended, ensure system reliability under expected conditions, and confirm that the platform meets stakeholder and client expectations. The results summarized below provide a comprehensive overview of system performance, highlighting the successful validation of key features and overall system readiness for deployment.

Feature Tested	Test Description	Expected Result	Actual Result	Test Result
Admin Login (FR-1)	Admin Logs in with valid credentials	Admin is authenticated and redirected to dashboard	Successful login and redirect	Pass
Create Event (FR-1)	Admin creates event with required fields only	Event is created and displayed	Event appears instantly on homepage	Pass
Set Date Range (FR-2)	Admin assigns event date range	Dates saved and displayed correctly	Dates display correctly	Pass
Event Priority	System assigns priority based on date	Events prioritized correctly	Priority reflected in	Pass

(FR-3)			UI	
Event Archiving (FR-4)	Past events are automatically archived/deleted	Event moved to archival/deletion	Events moved correctly	Pass
Flyer Upload (FR-5)	Admin upload flyer file	File stored and linked to event	Flyers displays correctly	Pass
Flyer Download (FR-5)	User downloads flyer PDF	File downloads and opens	Successful PDF download	Pass
Resource Link (FR-6)	User clicks Excel resource link	Correct document opens	Excel link open correctly	Pass
External Link (FR-7)	User clicks Mann-Grandstaff link	Redirects to correct page	Successful redirect	Pass
Event Tile UI (FR-8)	Event tiles render uniformly	All tiles display consistently	Uniform layout confirmed	Pass
Flyer Display (FR-9)	Click event to show flyer inline	Flyer appears without new tab	Works correctly	Pass
Email Link (FR-10)	User clicks email link	Opens default email client	Email draft opens	Pass
Search Function (FR-11)	User searches for event/resource	Relevant results displayed	Results returned correctly	Pass
Page Load Performance	Load homepage with events	Loads within 3 seconds	Averages 1.5 seconds return time	Pass
Responsive Design	Test on mobile/tablet/desktop	Layout flexes and adapts correctly	UI responsive	Pass
Accessibility	Keyboard navigation and screen reader	Fully navigable	Works correctly	Pass
UAT (Client Review)	Client reviews workflow	Meets expectations	Approved by client	Pass

All test cases passed successfully, indicating that the system meets both functional and non-functional requirements as defined in the Software Requirements Specification (SRS). Minor UI adjustments and performance optimizations were identified during testing and resolved prior to final deployment. User Acceptance Testing (UAT) confirmed that the system meets client expectations in terms of usability, accessibility, and functionality.

VI.1 Projects and Tools Used

Tool / Framework	Purpose in Project
Next.js 14	Full-stack React framework used for frontend rendering and API routes
React 18	Component-based UI rendering
Tailwind CSS	Utility-first CSS framework for consistent and responsive styling
HeroUI	UI component styling library
Prisma ORM	Type-safe database interaction and schema management
SQLite	Lightweight relational database for storing users, events, flyers, and sessions
Zod	Backend input validation and schema enforcement
<i>bcrypt</i>	Secure password hashing for authentication
NextAuth.js / JWT	Session management and authentication
Vercel	Deployment and CI/CD integration
GitHub	Version control and issue tracking
Figma	Wireframe and UI prototype design

VI.1. Languages Used in Project

The following languages were directly written or configured by the development team:

- JavaScript
- TypeScript
- HTML5
- CSS (Tailwind)
- SQL (SQLite)
- Markdown
- Bash (CLI commands for Prisma & deployment)

VII. Description of Final Prototype

The final prototype of the Whitman County Veteran e-Post Hub represents a fully functional web application deployed through Vercel and connected to a structured SQLite database via Prisma ORM. The system supports two primary user types: general users (veterans and community members) and administrative users.

VII.1. General User Experience

Landing Page

The landing page introduces the WCVS e-Post Hub and provides intuitive navigation to events, resources, and contact information. The layout emphasizes clarity and accessibility, following WCAG 2.1 AA guidelines.

Key features:

- Responsive navigation bar
- Featured event highlights
- Uniform event tile grid
- Persistent footer with contact links

Viewing Events

Users can:

1. Scroll to the Events section.
2. Click on any event tile.
3. View the associated flyer displayed within a fixed bounding container.
4. Download the flyer as a PDF file.

Event flyers:

1. Maintain consistent dimensions
2. Preserve aspect ratio
3. Display loading placeholders when necessary
4. Show “Flyer unavailable” message when no file exists

VII.2. Event Management (CRUD Operations)

The admin dashboard allows:

1. Create new events
2. Set date ranges
3. Assign priority levels

4. Upload flyer (PDF, JPG, PNG)
5. Edit existing events
6. Delete events (including associated flyer cleanup)
7. Live preview of event listings

All actions are:

1. Authenticated
2. Logged
3. Validated using Zod schema enforcement

VII.3. System Deployment

The application is deployed using Vercel with continuous integration enabled via GitHub repository synchronization. Every push to the main branch automatically triggers a rebuild and deployment process.

Deployment ensures:

1. HTTPS secure access
2. Global content distribution
3. Automatic database migration handling
4. Low operational overhead

VII.4. Brief User Manual

For General Users:

1. Navigate to the homepage.
2. Browse event listings.
3. Click event to view flyer.
4. Download flyer if desired.

For Administrators

1. Navigate to the admin login page.
2. Authenticate credentials.
3. Use dashboard panels to create, edit, or delete events.
4. Upload flyer file as needed.
5. Save changes and verify updates on the homepage.

VIII. Social Responsibility and Broader Impacts

VIII.1. Social Responsibility

This project was developed in alignment with the ACM Code of Ethics and Professional Conduct. As developers working on a public-service platform, we recognized the responsibility of building an accessible, secure, and inclusive system for a vulnerable and diverse user population.

Key ethical considerations included:

- Ensuring accessibility compliance (WCAG 2.1 AA)
- Protecting administrative credentials via encryption and role-based access
- Avoiding unnecessary collection of personal data
- Providing transparent and predictable system behavior

Given that the primary users include veterans of varying ages and technical backgrounds, usability and readability were treated as ethical priorities rather than aesthetic preferences.

VIII.2. Broader Impacts

This project directly benefits the Whitman County veteran community by:

- Centralizing critical information
- Reducing navigation complexity
- Improving digital accessibility
- Supporting local service outreach

The system also aligns with WSU EECS goals by:

- Applying real-world software engineering practices
- Demonstrating full-stack system design
- Promoting accessibility and inclusive technology
- Providing a maintainable foundation for future student teams

The project serves as a model for community-focused civic software and establishes a scalable architecture for potential future county-level expansion.

IX. Product Delivery Status

The final version A of the WCVS e-Post Hub (whitmanvs.com) was demonstrated to Veteran Service Officer Becky Buri and faculty mentor Dr. Parteek Kumar in December 2025 and February 2026.

Source Repository

GitHub Repository:

https://github.com/lsc-compsci/WCVS-Veteran_e-Post_Hub/tree/15-WCVS-branch

Deployment

Production Deployment: <https://wcvdeploy.vercel.app/>

Handoff Materials

As part of the final project delivery, all necessary materials required to maintain, deploy, and extend the system were transferred to the client and faculty mentor. These materials include the complete Prisma schema file to document the database structure and support future migrations, an environment configuration template outlining required environment variables for secure setup, and detailed deployment instructions describing the build and hosting workflow. Administrative login credentials were transferred securely to the client to ensure authorized access to event management functionality. Together, these materials provide sufficient documentation and technical resources for future developers or administrators to rebuild, maintain, and operate the system independently.

Rebuild Instructions

To rebuild the system:

1. Clone repository.
2. Install dependencies using `npm install`.
3. Configure `.env` file.
4. Run `npx prisma migrate deploy`.
5. Start server with `npm run dev`.

All required setup instructions are included within the repository README file.

X. Conclusions and Future Work

The following work items outline the major tasks that we hope to achieve for Spring 2026, and the future development of this website, and the steps required to reach Beta and Final Release quality.

1. Complete Authentication and Authorization

Implement full session control using secure JWT cookies or NextAuth.js to manage authentication reliably across the platform. All administrative routes must strictly enforce role-based access control to prevent unauthorized actions. Additionally, session expiration, logout functionality, and audit logging should be added to meet non-functional requirements for security and accountability.

2. Implement Full File Storage Solution

Integrate Vercel Blob or AWS S3 to support permanent and reliable storage of event flyers. Enable a robust, in-browser PDF preview on event pages to improve accessibility and user experience. Add automatic thumbnail generation to ensure all event tiles display with consistent, uniform visuals.

3. Expand Event Management + Interaction

Implement edit and delete actions for events to support full lifecycle management. Add automatic archiving for past events in alignment with FR-4, ensuring outdated events are removed from active views but remain accessible historically. Implement event prioritization logic per FR-3 so that higher-priority or upcoming events are surfaced more prominently in the interface.

4. Add Search and Navigation Enhancements

Implement FR-11 by adding keyword search for events and resources to help users quickly find relevant content. Improve the navigation flow between event pages and resource directories so users can move seamlessly between related information. Add breadcrumbs and filter options to provide clearer context and more precise control over how events and resources are browsed.

5. Resource Management System

Integrate the Excel sheet link dynamically as specified in FR-6 so that updates are reflected automatically. Build structured resource cards grouped by clear categories to make information easier to browse and understand. Ensure all external links open correctly and comply with accessibility standards, including proper labels and behaviors for assistive technologies.

6. Comprehensive Testing Expansion

Write comprehensive Vitest test suites for all components to ensure reliable behavior and regression coverage. Develop Selenium scripts that simulate real Veterans Service Officer (VSO) workflows to validate end-to-end functionality in a realistic context. Conduct a formal

WCAG 2.2 accessibility audit to verify compliance and identify any usability barriers for users with disabilities.

7. Performance and Deployment Enhancements

Improve API response times by optimizing database queries and schema with Prisma. Migrate from SQLite to PostgreSQL for production to support better scalability, reliability, and concurrency. Prepare a scalable deployment pipeline using GitHub Actions to automate testing, builds, and deployments for the application.

8. Final Client Preparation

Polish the user interface to ensure clarity, visual consistency, and a cohesive user experience across all views. Prepare clear onboarding documentation for the VSO to support efficient adoption and daily use of the system. Ensure the mobile interface matches the desktop experience in functionality and usability. Produce the final project poster, comprehensive report, and presentation materials for formal delivery and evaluation.

X.1. Limitations and Recommendations

The current prototype, while fully functional for its intended scope, presents several limitations that should be addressed in future iterations. First, the use of SQLite as the primary database solution introduces scalability constraints, particularly if the platform experiences increased traffic or requires concurrent multi-user administrative access. Although SQLite is efficient and lightweight for small-scale deployments, it is not optimized for high-concurrency production environments. Additionally, the system currently supports only a single administrative role without granular permission controls, limiting flexibility in scenarios where multiple administrators may require differentiated access levels. The search functionality, while operational, may require further optimization to improve performance and efficiency as the volume of stored events and resources increases. Finally, the system does not currently include an automated mechanism for collecting end-user feedback or usage analytics, which limits insight into user behavior and system effectiveness.

To address these limitations, several improvements are recommended. Migrating the database from SQLite to PostgreSQL would provide enhanced scalability, improved concurrency handling, and better long-term production support. Implementing role-based access control would allow differentiated administrative permissions, increasing security and operational flexibility. Introducing full-text search indexing would significantly improve search performance and result relevance as the dataset grows. Finally, integrating analytics tools would enable data-driven decision-making by providing visibility into user engagement patterns and system usage trends. Together, these enhancements would strengthen the system's robustness, scalability, and maintainability.

X.2. Future Work

Future development of the Whitman County Veterans e-Post Hub may focus on expanding functionality, scalability, and long-term sustainability. One potential enhancement includes implementing a multi-role administrative hierarchy, allowing differentiated permission levels for various administrators and improving operational flexibility. Cloud storage integration, such as AWS S3 or Vercel Blob, would provide more scalable and reliable file management for event flyers and related documents. Additional automation features, including email notifications for new or updated events, could further improve user engagement and communication efficiency. Enhancing the search functionality through improved indexing and filtering mechanisms would allow users to locate relevant information more quickly as the platform grows. A mobile-first user interface optimization strategy would ensure improved accessibility and usability for veterans accessing the system from smartphones and tablets. Finally, expansion beyond Whitman County to support additional counties or regional veteran service organizations represents a long-term opportunity to scale the platform's impact and broaden its community reach.

XI. Acknowledgements

We would like to express our sincere gratitude to everyone who contributed to the successful completion of this project. First and foremost, we would like to thank Veteran Service Officer Becky Buri for her continuous support, guidance, and valuable insights into the needs of the veteran community. Her feedback played a crucial role in shaping the direction and goals of this project.

We are also deeply grateful to Dr. Parteek Kumar for his mentorship, technical guidance, and encouragement throughout the development process. His expertise and advice were instrumental in helping us navigate challenges and improve the overall quality of the project.

Additionally, we would like to acknowledge the efforts of previous developers who contributed to the foundation of the WCVS e-Post platform. Their work provided a strong starting point that made further improvements and enhancements possible.

Finally, we would like to thank our team members for their collaboration, dedication, and consistent effort throughout the project. The collective contributions of the group were essential in ensuring that the project meets both technical standards and the needs of its users.

XII. Glossary

Control Management System - A software tool that allows users to create, edit, and publish content through an easy-to-use interface, without needing to write any code.

Digital Accessibility - ensuring content and technologies are usable by everyone, including people with various disabilities, by removing barriers and promoting equal access to information and services.

Web Development - process of creating and maintaining websites and web applications; combining technical skills and creative problem-solving to build functional, interactive digital experiences.

Functional Requirements — Define what a system must accomplish by specifying the features, services, and behaviors that directly support user tasks and interactions. They describe the

explicit functions the system is expected to perform.

Non-Functional Requirements — Define how a system should operate by establishing quality attributes, performance criteria, and constraints. They address characteristics such as usability, reliability, efficiency, and maintainability, rather than specific system behaviors.

Front-End — Refers to the client-facing portion of a software system, typically encompassing the user interface (UI) and user experience (UX). The front-end is responsible for presenting information, capturing user input, and ensuring accessibility and usability. For this project, the front-end design was developed using NextUI (now referred to as HeroUI), a React-based component library that supports responsive and accessible interface design.

Back-End — Refers to the server-side portion of a software system, responsible for implementing business logic, processing data, and managing communication with databases or external services. The back-end ensures system reliability, scalability, and security while supporting the functionality delivered through the front-end. It is typically implemented using server-side programming languages, frameworks, and database technologies. For this project, the back-end is developed using Prisma (an open-source ORM for database management) in conjunction with SQLite queries to handle structured data storage and retrieval

CRUD - Abbreviation for Create, Read, Update, Delete. In the context of this project CRUD operations are handled by API routes: POST, GET, PUT, DELETE respectively.

Zod - Typescript-first validation library used to ensure data requests and responses (e.g., from user forms or API requests) has the correct structure and type before it is processed or stored in the database.

bcrypt - password-hashing algorithm designed to securely store user passwords. It transforms strings into irreversible, random hashes that are safe to store in the database.

Administrative dashboard – The secure interface that allows the Veterans Service Officer (VSO) to manage events, review submissions, and update resource information.

Automated testing – Testing performed using tools such as Vitest and Selenium to verify system behavior without manual execution.

Black box testing – A testing method that evaluates functional behavior without inspecting the internal code or implementation.

Bug – Any defect, failure, or unexpected behavior identified during testing that requires correction.

CI/CD (Continuous Integration / Continuous Delivery) – A workflow practice in which changes are routinely tested and validated before being merged to ensure system stability.

Component – A small, isolated functional unit within the application, such as a UI element or utility function, that can be individually tested.

Defect – An inconsistency between expected and actual behavior discovered during testing.

End-to-end testing – A full workflow test evaluating the application from the user's perspective through the entire system.

Functional testing – Testing that verifies features and behaviors directly against the defined requirements.

Integration testing – Testing focused on interactions between multiple components or modules to confirm proper data flow and communication.

Issue-tracking system – The team's shared Kanban board is used to document bugs, failed test cases, resolutions, and testing tasks.

Performance testing – Testing that measures responsiveness, load behavior, and system stability under normal and stressed conditions.

Prisma – The database ORM used to manage data models, handle queries, and interact with stored records during testing.

Responsive design – A design approach ensuring that the application layout adapts correctly across different screens and device sizes.

Selenium WebDriver – The browser automation tool used to conduct system-level and acceptance testing.

Staging environment – A controlled environment used to test the integrated system prior to deployment.

System testing – A testing phase that evaluates the entire application functioning as one cohesive system.

Test case – A defined set of inputs, execution steps, and expected outputs used to verify correct system behavior.

Test configuration – The specific environment, data, tools, and software settings under which testing is executed.

Test data – Structured data used to drive test cases and confirm expected outcomes.

Unit testing – Testing individual, isolated components or logic segments to verify correctness.

User Acceptance Testing (UAT) – Final evaluation performed by the client (VSO) to confirm that the system satisfies operational requirements.

Vitest – The testing framework used for both unit and integration testing of the project's frontend and backend logic.

XIII. References

- [1] “Welcome to Whitman County Veteran page,” Veteran e-Post Hub, <https://whitmancountyveterans.com/> (accessed Sep. 6, 2025).
- [2] U.S. Department of Veterans Affairs, “VA.gov,” U.S. Department of Veterans Affairs, 2025. [Online]. Available: <https://www.va.gov>. [Accessed: 8- Sept- 2025].
- [3] Vercel. 2024. “Getting Started | Next.js.” Nextjs.org. 2024. <https://nextjs.org/docs>.
- [4] Prisma, “Prisma Documentation,” Prisma.io, 2025. [Online]. Available: <https://www.prisma.io/docs/>. [Accessed: 10- Sept- 2025].
- [5] “SQLite Documentation.” n.d. Sqlite.org. <https://sqlite.org/docs.html>.
- [6] “Vercel – Introduction to Vercel.” n.d. Vercel Documentation. <https://vercel.com/docs>.
- [7] “Software Engineering | Software Evolution.” GeeksforGeeks, www.geeksforgeeks.org/software-engineering/software-engineering-software-evolution. (Accessed 27 Sept. 2025).
- [8] Tailwind CSS, Tailwind CSS Documentation. [Online]. Available: <https://tailwindcss.com/docs>
- [9] A. Tailor, V. Shrivastava, A. Pandey, and S. Kumar, “Unraveling Next.js: A comprehensive overview,” Int. J. Res. Publ. Rev., vol. 5, no. 3, pp. 2675-2679, Mar. 2024. [Online]. Available: <https://ijrpr.com/uploads/V5ISSUE3/IJRPR23656.pdf>
- [10] J. Ajayi, “Prisma ORM adoption guide: Overview, examples, and alternatives,” LogRocket Blog, Sep. 2024. [Online]. Available: <https://blog.logrocket.com/prisma-orm-adoption-guide/>
- [11] Vitest Team, Vitest: A Vite-Native Unit Testing Framework. [Online]. Available: <https://vitest.dev/>
- [12] Selenium Project, *Selenium WebDriver Documentation*. [Online]. Available: <https://www.selenium.dev/documentation/webdriver/>
- [13] ISO/IEC/IEEE 29119-1:2022, Software and Systems Engineering — Software Testing — Part 1: Concepts and Definitions, 2022. [Online]. <https://www.iso.org/standard/80903.html>

[14] IEEE, *IEEE Std 829-2008 – Standard for Software and System Test Documentation*, 2008. [Online]. <https://standards.ieee.org/standard/829-2008.html>

[15] W3C, Web Content Accessibility Guidelines (WCAG) 2.2, 2023. [Online]. <https://www.w3.org/TR/WCAG22/>

[16] ACM, ACM Code of Ethics and Professional Conduct. [Online]. <https://www.acm.org/code-of-ethics>

[17] European Union, General Data Protection Regulation (GDPR) – Regulation (EU) 2016/679, 2018. [Online]. <https://gdpr.eu/>

Appendix A - Mockups and Refined UI Examples

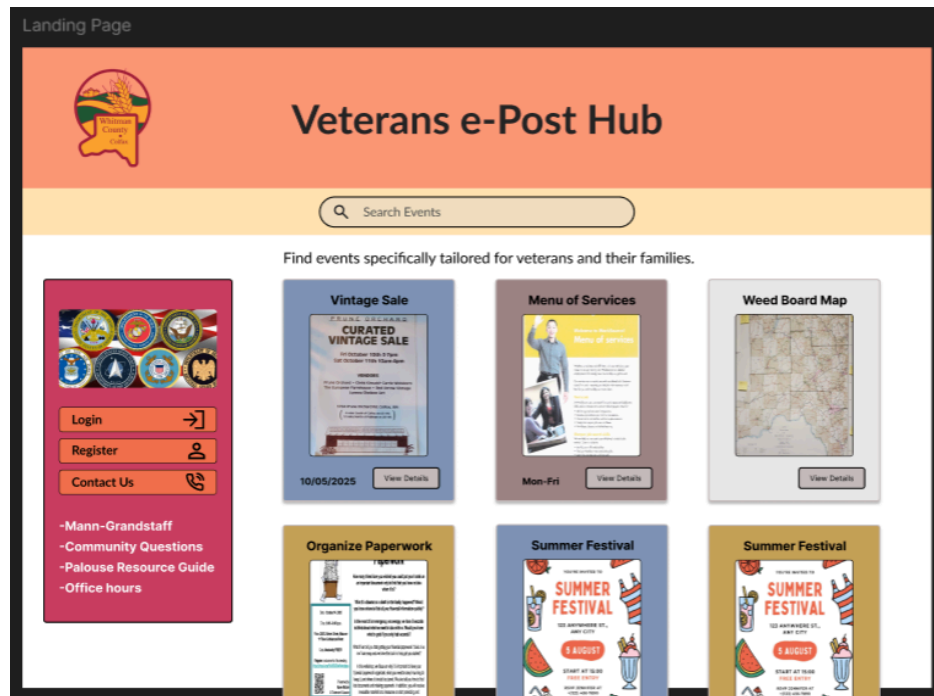


Figure 8. Landing Page Mockup for Desktop UI

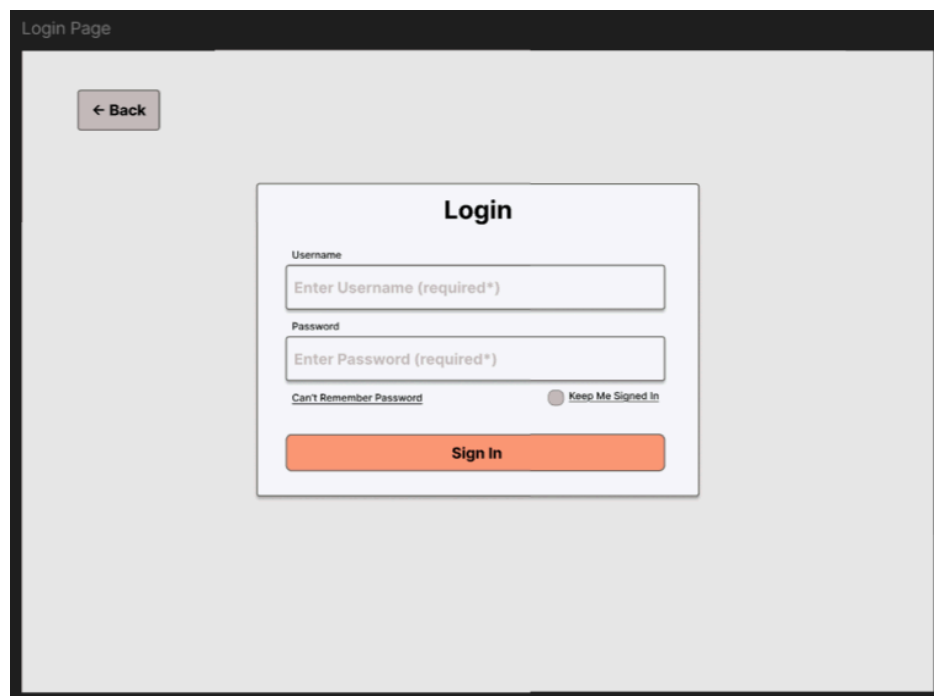


Figure 9. Login Page Mockup for Admin on Desktop UI

Create New Event Page

← Back

Create New Event

Enter Event Title (Required*)

+ Add Dates

Enter Event Description (Optional)

Flyer / Attachment (Required*)

Choose File No File Selected

Event Address (Optional)

Event Address (Optional)

Submit Event

Figure 10. Create Event page Mockup for Admin on Desktop UI

Home - Landing Page

Veterans e-Post Hub

Resources Community Questions Login

Welcome to the Whitman County Veterans Page

Find events specifically tailored for veterans and their families.

Search Events

17th Annual Autumn Festival
Saturday, October 4th & Sunday, October 5th

Event One [more info](#)

HALLOWEEN CARNIVAL
FREE ADMISSION!
October 21st

Event two [more info](#)

Login

Veterans e-Post Hub

Resources Community Questions Login

← Back

Welcome, user.

Username

Enter username

Password

Enter password

[Can't remember password](#) [Create an account](#)

login

Figure 11. Landing page and Login page Mockup for Admin, respectively for Mobile UI.

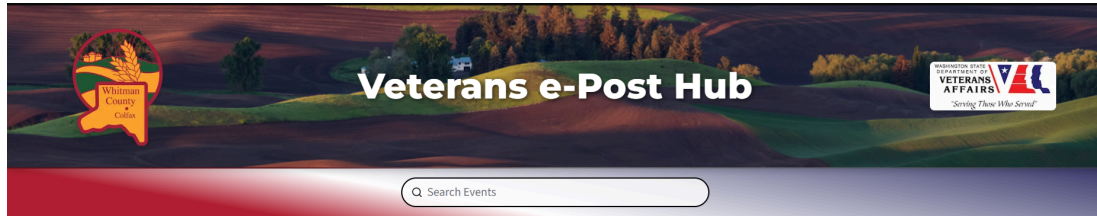


Figure 12. Banner Re-design with added Search function.

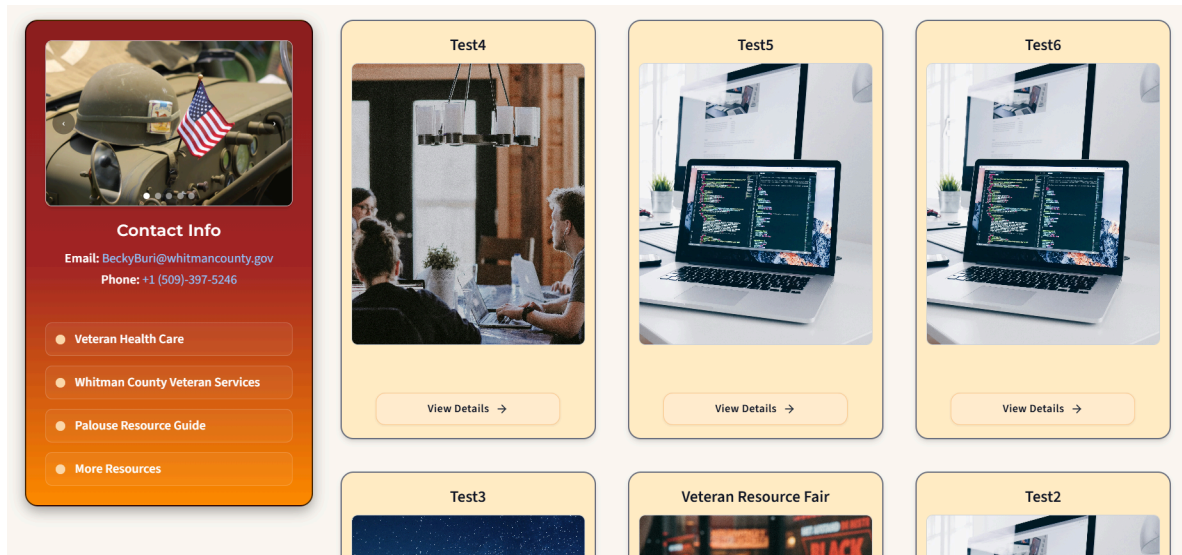


Figure 13. New Page Layout with improved accessibility + visual features.

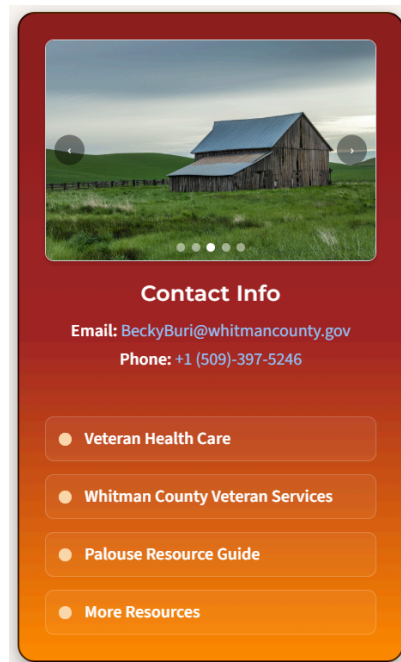


Figure 14. Side bar with Resource quick-links (responsive buttons) + image slideshow.

Appendix B - Team Information

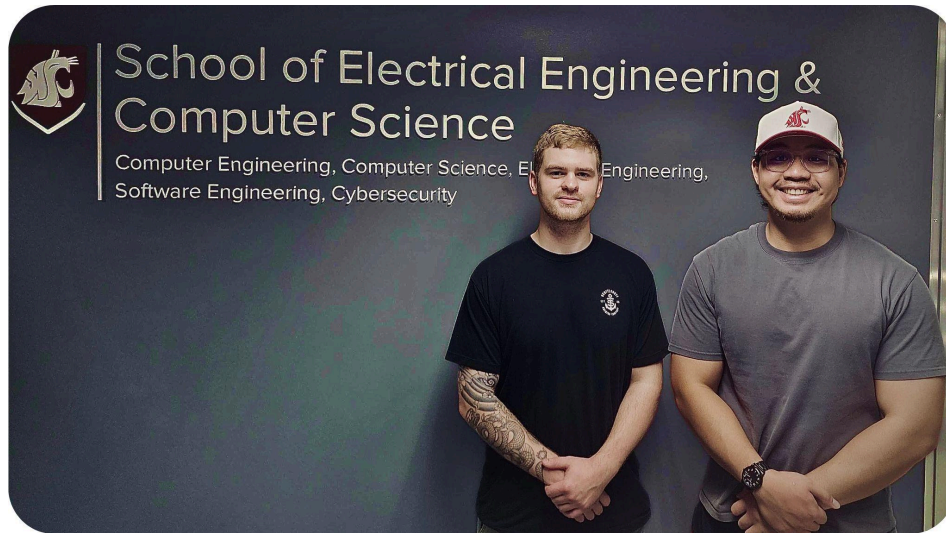


Figure 15. Developers - Logan Cribbs and Chhi Ly, respectively.

Appendix C - Example Testing Strategy Reporting

```
try:
    # Step 1: Open the login page
    driver.get("http://host.docker.internal:3000/Login") # Use host.docker.internal to access the app from Docker
    time.sleep(2) # Wait for page to load

    # Step 2: Locate the email input and enter email
    email_input = driver.find_element(By.NAME, "email")
    email_input.send_keys("john@test.com")

    # Step 3: Locate the password input and enter password
    password_input = driver.find_element(By.NAME, "password")
    password_input.send_keys("password")

    # Step 4: Find the login button and click it
    login_button = driver.find_element(By.XPATH, "//button[text()='Login']")
    login_button.click()

    # Step 5: Wait for login process and verify success
    time.sleep(3) # Give some time for the login action to complete

    # Verifying by URL
    current_url = driver.current_url
    if "/Login" in current_url:
        print("Test Passed: User successfully redirected to /Member.")
    else:
        print("Test Failed: User was not redirected to /Member.")

finally:
    # Step 6: Close the browser
    driver.quit()
```

Figure 16. Automated “Login” Use Case Selenium for Test Example.

Appendix D - Project Management

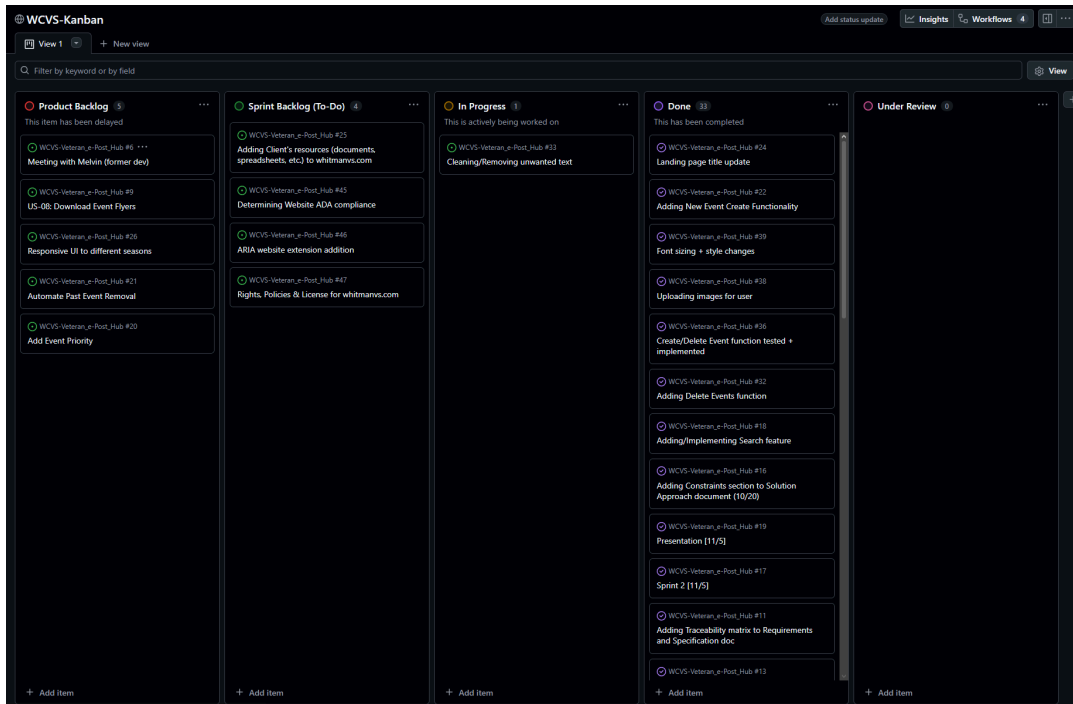


Figure 17. WCVS Kanban Board - Agile Development.

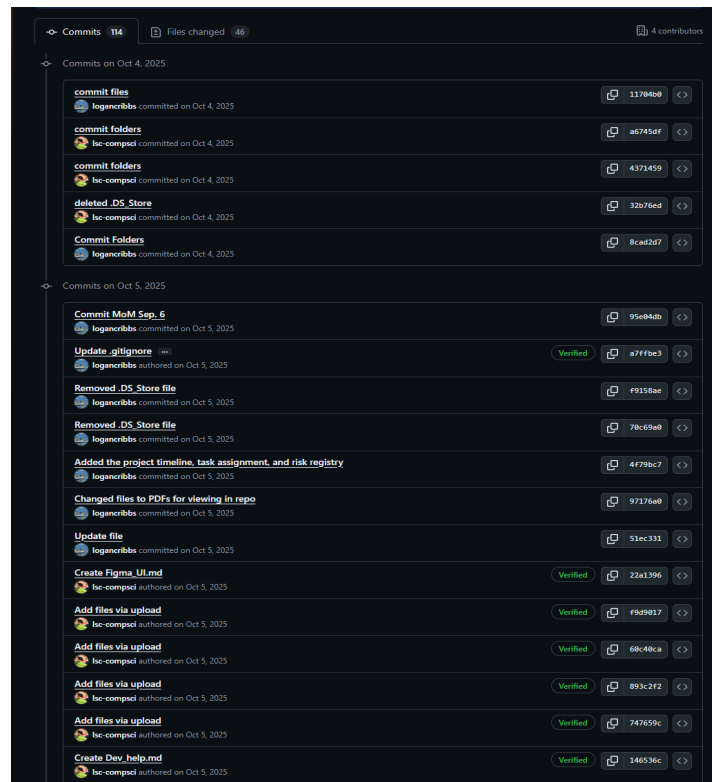


Figure 18. Github Repository Commit History for Effective Logging.